

1. Consider the pattern $P=209$. Using the “slow” pattern matching algorithm, find the INDEX and the number C of comparisons to find the INDEX of P in each of the following texts T .

[for 3 to 4 marks]

- 2920209208
- 69112052243
- 321432091134

♣ **Ans:**

- a. We know, in slow pattern matching,

$$\text{The number of substrings} = \text{Length}(T) - \text{Length}(P) + 1$$

$$= 10 - 3 + 1 = 8$$

As, P has 3 characters, we must create substrings with 3 characters from T .

Index of T	Substrings of T	P	Mismatched Index
1	292	209	2
2	920	209	2
3	202	209	3
4	020	209	1
5	209	209	3
C (number of Comparisons) =			11

In the string T , P is found from the INDEX number 5.

And number of comparisons $C = 11$.

2. Consider the pattern $P = abc$. Using the “slow” pattern matching algorithm, find the INDEX and the number C of comparisons to find the INDEX of P in each of the following texts T .

[for 3 to 4 marks]

- $(ab)^5$
- $(cbab)^5$

♣ **Ans:**

- a. $\text{Length}(T) = (ab)^5 = ababababab = 10$

$$\text{Length}(P) = 3$$

We know, in slow pattern matching,

$$\text{The number of substrings} = \text{Length}(T) - \text{Length}(P) + 1 = 10 - 3 + 1 = 8$$

As, P has 3 characters, we must create substrings with 3 characters from T .

Index of T	Substrings of T	P	Mismatched Index
1	aba	abc	3
2	bab	abc	1
3	aba	abc	3
4	bab	abc	1
5	aba	abc	3
6	bab	abc	1
7	aba	abc	3
8	bab	abc	1
C (number of Comparisons) =			16

In T there is no P found.

b. $\text{Length}(T) = (\text{cbab})^5 = \text{cbabcbabcbabcbabcbab} = 20$

$\text{Length}(P) = 3$

We know, in slow pattern matching,

$$\text{The number of substrings} = \text{Length}(T) - \text{Length}(P) + 1$$

$$= 20 - 3 + 1 = 18$$

As, P has 3 characters, we must create substrings with 3 characters from T.

Index of T	Substrings of T	P	Mismatched Index
1	cba	abc	1
2	bab	abc	1
3	abc	abc	3
	C =		5

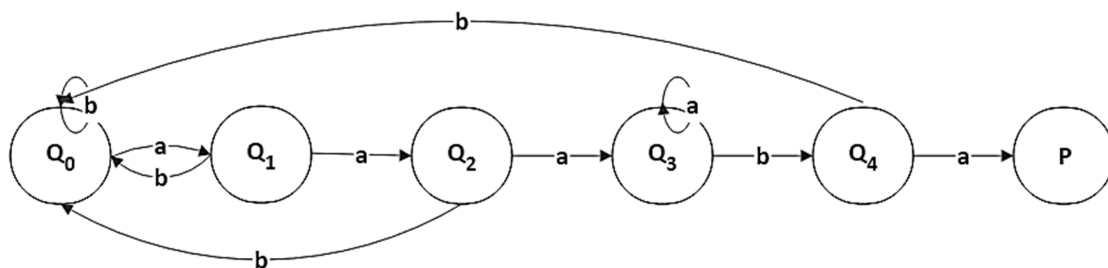
In the string T, P is found from the INDEX number 3.

And number of comparisons $C = 5$

- For each of following pattern's P and texts T, find the number of C of comparisons to find INDEX of P in T using slow or first pattern matching algorithm: *[for 3 to 4 marks]*
 - $P = \text{qrs}, T = (\text{qr})^r$
 - $P = \text{ccc}, T = \text{cdceddcccddcccddddd}$
- Using first pattern matching algorithm find number of C of comparisons to find INDEX of P in T. $P = \text{xyz}, T = (\text{xy})^5$. *[for 3 to 4 marks]*
- Consider the pattern $P = \text{a}^3\text{ba}$. Construct the table and the corresponding labeled directed graph used in "fast" pattern matching algorithm. *[for 4 marks]*

♣ Ans:

Q		a	b
Q0	Σ	Q1	Q0
Q1	a	Q2	Q0
Q2	aa	Q3	Q0
Q3	aaa	Q3	Q4
Q4	aaab	P	Q0



- Give the differences between first (slow) and second (fast) pattern matching algorithm.

♣ Ans:

Here are the key comparisons between the Naive String-Matching algorithm and the Knuth-Morris-Pratt (KMP) algorithm:

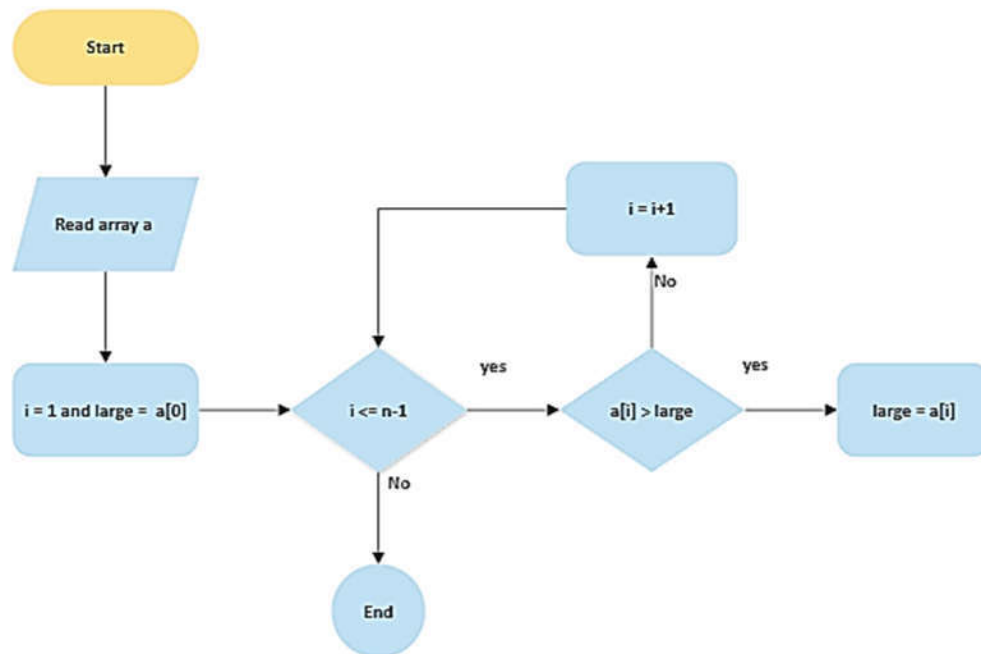
Criteria	First or Slow	Second or Fast
Time Complexity	Naive String Matching: $O(n * m)$ Quadratic time complexity	KMP Algorithm: $O(n + m)$ Linear time complexity
Approach	Naive String Matching: Compares the pattern with each substring of the text, character by character	KMP Algorithm: Utilizes partial match information within the pattern to avoid unnecessary comparisons
Preprocessing	Naive String Matching: No preprocessing step is involved	KMP Algorithm: Constructs a prefix table or "failure function" based on the pattern to optimize matching
Efficiency	Naive String Matching: Can result in a high number of unnecessary comparisons, making it inefficient for large texts or patterns	KMP Algorithm: Significantly reduces the number of comparisons required, making it more efficient for pattern matching tasks

◆ **Flow chart**

[for 2 to 2.5 marks]

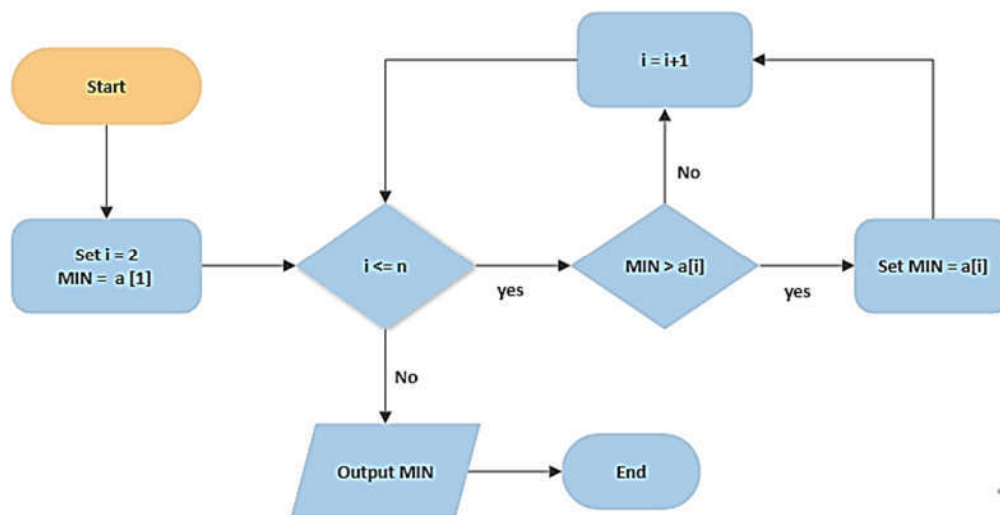
1. Draw a flowchart to find the largest element of an array

♣ **Ans:**



2. Draw a flow chart to find the smallest value MIN from a non-empty array DATA with N numerical values.

♣ **Ans:**



3. Draw a flowchart to solve the quadratic equation $ax^2 + bx + c = 0$

◆ **Theory questions** *[for 2 to 2.5 marks]*

1. What do you mean by data? What is data structure. Give difference b/w data and information.
2. Abstract data type? Big O notation?
3. What is control structure? How many types of control structure? Describe with example.

♣ **Ans:**

The structure which controls the flow of an algorithm and their equivalent computer programs are called control structure.

There are three types of CS.

Sequence Logic

Module A



Module B



Module C

Selection Logic

1. Single Alternative

if (condition) then:
Module A

2. Double Alternative

if (condition) then:
Module A
else

Module B

3. Multiple Alternative

if (condition) then:
Module A
else if (condition 1) then:
Module B

.....

else

Module N

Iteration Logic

1. Repeat for loop

Repeat for I = X to Y by T
Module A

2. Repeat while loop

Repeat while condition
Module A

4. Write the steps of study of subject data structure

♣ **Ans:**

The logical or mathematical model of a particular organization of data is called data structure.

Classifications of Data structure:

1. Linear Data structure
 - a. Array
 - b. Linked List
 - c. Stack
 - d. Queue
2. Non Linear Data structure
 - a. Graph
 - b. Tree

Data structure Operations:

1. Traversing
2. Searching
3. Insertion
4. Deletion
5. Sorting
6. Merging

Segment 2

◆ Linear Array

1. Suppose STAD maintains a linear array STUCGPA (2100: 2199) such that, STUCGPA [K] contains the CGPA of the students who bears the ID number K.

Write a module for each of the following tasks: *[Usually given for 3 marks]*

1. Print the ID number of the students who have CGPA above 3.75
2. Count the number of students who have CGPA equal to or above 3.5

♣ Ans:

1.
 - i. Set $n = 100$
 - ii. Repeat for $K = 1$ to $K \leq n$
 - iii. If (STUCGPA[K] > 3.75)
 - iv. Write: K
 - v. Exit

2.
 - i. Set $n = 100$, count = 0
 - ii. Repeat for $K = 1$ to $K \leq n$
 - iii. If (STUCGPA[K] >= 3.75)
 - iv. Count = Count + 1
 - v. Write: Count
 - vi. Exit

2. Write an algorithm to *insert* an element ITEM from the K^{th} position of a linear array LA. *[2.5]*

♣ Ans:

- i. Initialize an array A of size $n+1$
- ii. Copy the first $K-1$ elements of the original array into the new array A
- iii. Insert ITEM into the Kth position of A
- iv. Copy the remaining elements of the original array into A, starting from the $(K+1)^{\text{th}}$ position
- v. The new array A now contains the original array with the ITEM inserted at the Kth position

3. An array A contains 25 positive integers. Write a module which finds the number EVNUM of elements of A which are even, and the number ODDNUM of elements of A which are odd.

♣ Ans: a.

- i. Set $n = 25$
- ii. Repeat for $K = 1$ to $K \leq n$
- iii. If ($A[K] \% 2 = 0$)
- iv. Set EVNUM = EVNUM + 1
- v. Else
- vi. Set ODDNUM = ODDNUM + 1
- vii. Write: EVNUM, ODDNUM
- viii. Exit

4. Consider the linear arrays XXX (-10:10), YYY (1935:1985), ZZ (35)

1. Find the number of elements in each array

2. Suppose $Base(YYY) = 400$ and $w = 4$ words per memory cell for YYY. Find the address of YYY [1942], YYY [1977].

♣ Ans:

1. Number of elements in XXX array:

first element = -10

Last element = 10

Total elements = $10 - (-10) + 1 = 21$

- Number of elements in YYY array:

First element = 1935

Last element = 1985

Total elements = $1985 - 1935 + 1 = 51$

- Number of elements in ZZ array:

First element = 35

Last element = 35

Total elements = $35 - 35 + 1 = 1$

2. $Base(YYY) = 400$, $w = 4$

Address YYY [1942] = $Base + (1942 - 1935 + 1) * 4 = 400 + 32 = 432$

Address YYY [1977] = $Base + (1977 - 1935 + 1) * 4 = 400 + 168 = 568$

5. What is linear array?

♣ Ans:

A linear array is a list of a finite number of n homogeneous data elements (i.e., data elements of the same type) such that:

- The elements of the array are referenced respectively by an index set consisting of n consecutive numbers
- The elements of the array are stored respectively in such memory locations.

6. What are the disadvantages of using linear array?

♣ Ans:

- We must know beforehand how many elements will be there in the array.
- There are chances of wastage of memory space if elements inserted in an array are lesser than the allocated size.
- Insertion and deletion become tedious.
- All of the mentioned.

◆ Linear Search

1. Given two linear search algorithms in the following. Determine which one is efficient and why? [2 to 3 marks]

(Linear Search) A linear array DATA with N elements and a specific ITEM of information are given. This algorithm finds the location LOC of ITEM in the array DATA or sets LOC = 0.

1. [Initialize] Set K := 1 and LOC := 0.
2. Repeat Steps 3 and 4 while LOC = 0 and K ≤ N.
3. If ITEM = DATA[K], then: Set LOC := K.
4. Set K := K + 1. [Increments counter.]
[End of Step 2 loop.]
5. [Successful?]
If LOC = 0, then:
Write: ITEM is not in the array DATA.
Else:
Write: LOC is the location of ITEM.
[End of If structure.]
6. Exit.

(Linear Search) LINEAR(DATA, N, ITEM, LOC)

Here DATA is a linear array with N elements, and ITEM is a given item of information. This algorithm finds the location LOC of ITEM in DATA, or sets LOC := 0 if the search is unsuccessful.

1. [Insert ITEM at the end of DATA.] Set DATA[N + 1] := ITEM.
2. [Initialize counter.] Set LOC := 1.
3. [Search for ITEM.]
Repeat while DATA[LOC] ≠ ITEM:
Set LOC := LOC + 1.
[End of loop.]
4. [Successful?] If LOC = N + 1, then: Set LOC := 0.
5. Exit.

♣ Ans:

In terms of time complexity both algorithm is same. However, in second algorithm has no extra variable to declare for increment counter. In that case, second algorithm is efficient.

◆ Bubble sort

Code: <https://onlinegdb.com/gsjH-3jSmS>

1. Sort the following list of numbers using Bubble sort.

[Usually given for 4 to 5 marks]

44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, 66

♣ Ans:

Pass 1:

- 1) **44 33** 11 55 77 90 40 60 99 22 88 66
- 2) 33 **44 11** 55 77 90 40 60 99 22 88 66
- 3) 33 11 44 55 77 **90 40** 60 99 22 88 66
- 4) 33 11 44 55 77 40 **90 60** 99 22 88 66
- 5) 33 11 44 55 77 40 60 90 **99 22** 88 66
- 6) 33 11 44 55 77 40 60 90 22 **99 88** 66
- 7) 33 11 44 55 77 40 60 90 22 88 **99 66**

Pass 2:

- 8) **33 11** 44 55 77 40 60 90 22 88 66 99
- 9) 11 33 44 55 **77 40** 60 90 22 88 66 99
- 10) 11 33 44 55 40 **77 60** 90 22 88 66 99
- 11) 11 33 44 55 40 60 77 **90 22** 88 66 99
- 12) 11 33 44 55 40 60 77 22 **90 88** 66 99
- 13) 11 33 44 55 40 60 77 22 88 **90 66** 99

Pass 3:

14) 11 33 44 **55 40** 60 77 22 88 66 90 99

15) 11 33 44 40 55 60 **77 22** 88 66 90 99

16) 11 33 44 40 55 60 22 77 **88 66** 90 99

Pass 4:

17) 11 33 **44 40** 55 60 22 77 66 88 90 99

18) 11 33 40 44 55 **60 22** 77 66 88 90 99

19) 11 33 40 44 55 22 60 **77 66** 88 90 99

Pass 5:

20) 11 33 40 44 **55 22** 60 66 77 88 90 99

Pass 6:

21) 11 33 40 **44 22** 55 60 66 77 88 90 99

Pass 7:

22) 11 33 **40 22** 44 55 60 66 77 88 90 99

Pass 8:

23) 11 **33 22** 40 44 55 60 66 77 88 90 99

Sorted array:

11 22 33 40 44 55 60 66 77 88 90 99

2. Given a string SIBPLE, find the number of *comparisons*(C) and number of *interchanges*(D) needed to sort the string alphabetically (A to Z) by using bubble sort algorithm. [3 marks]

♣ **Ans:**

$$\text{Comparisons (C)} = \frac{n(n-1)}{2} = \frac{6 \cdot 5}{2} = 15$$

$$\text{Interchanges(D)} = 17$$

3. Using bubble sort algorithm, find the number C of comparisons and the number D of interchanges which alphabetize the n = 6 letters in IIUCSE. [Usually given for 4 marks]

♣ **Ans:**

$$\text{Comparisons (C)} = 21$$

$$\text{Interchanges(D)} = 14$$

4. Write bubble sort algorithm. Analyze the complexity of bubble sort. [for 2 to 3 marks]

♦ **Binary Search**

[Binary Search - Data Structure and Algorithm Tutorials - GeeksforGeeks](https://www.geeksforgeeks.org/binary-search-data-structure-and-algorithm-tutorials/)

1. Search the ITEM = 88 from the following list of numbers using Binary search algorithm. {11, 22, 33, 44, 55, 66, 77, 88, 99} Show each step. [Usually given for 3 marks]

♣ **Ans:**

For ITEM = 88

(1) Initially, Beg = 1 and End = 9

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (1+9)/2 = 5$$

$$\text{Data}[\text{Mid}] = 55$$

11, 22, 33, 44, **55**, 66, 77, 88, 99

(2) Since, $88 > 55$,

$$\text{Beg} = \text{Mid} + 1 = 5 + 1 = 6 \text{ and } \text{End} = 9$$

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (6+9)/2 = 7$$

$$\text{Data}[\text{Mid}] = 77$$

11, 22, 33, 44, 55, 66, **77**, 88, 99

(2) Since, $88 > 77$,

$$\text{Beg} = \text{Mid} + 1 = 7 + 1 = 8 \text{ and } \text{End} = 9$$

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (8+9)/2 = 8$$

$$\text{Data}[\text{Mid}] = 88$$

11, 22, 33, 44, 55, 66, 77, **88**, 99

We've found ITEM in LOC = Mid = 8

2. Let DATA be the following sorted 13 elements array:

[Usually given for 3 to 4 marks]

DATA: 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99

Apply the binary search to DATA for ITEM = 40

♣ Ans:

For ITEM = 40

(1) Initially, Beg = 1 and End = 13

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (1+13)/2 = 7$$

$$\text{Data}[\text{Mid}] = 55$$

(2) Since, $40 < 55$, End = Mid - 1 = 6

$$\text{Beg} = 1$$

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (1+6)/2 = 3$$

$$\text{Data}[\text{Mid}] = 30$$

(2) Since, $40 > 30$,

$$\text{Beg} = \text{Mid} + 1 = 4 \text{ and } \text{End} = 6$$

$$\text{Mid} = (\text{Beg} + \text{End})/2 = (4+6)/2 = 5$$

$$\text{Data}[\text{Mid}] = 40$$

We've found ITEM in LOC = Mid = 5

♦ 2D array/ Multidimensional array/ Matrix/ Algebra of matrices

1. How a 2D array is represented in computer memory? Explain with example.

♣ Ans:

There are two ways to represent a 2D array in computer memory.

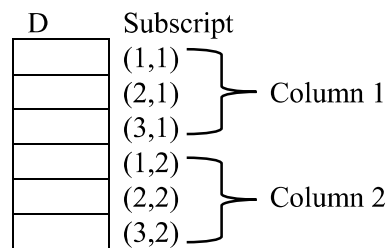
- i. Column major order
- ii. Row major order

Let us consider a 2D array D [3][2] with 3 rows and 2 columns.

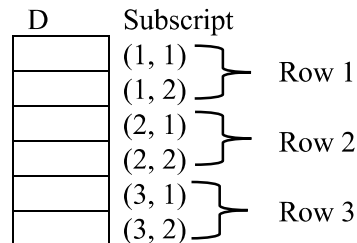
	C 1	C 2
R 1		
R 2		
R 3		

It can be represented in two ways.

Column major order: In this order, elements are stored in memory column by column.



Row major order: In this order, elements are stored in memory row by row.



Address in two-dimensional array:

In column major order, address of any element A [J, K] is,

$$LOC(A [J, K]) = Base (A) + w[R (K - LB_C) + (J - LB_R)]$$

And in row major order, address of any element A [J, K] is,

$$LOC(A [J, K]) = Base (A) + w[C(J - LB_R) + (K - LB_C)]$$

Where, Base (A) = Base address of A

R = Rows of A

C = Columns of A

LB_R = Lower bound of rows

LB_C = Lower bound of columns

W = Number of words per memory location

2. Consider the Multidimensional array A (-5:5, 3:33) and B (3:10, 1:15, 10:20) to find the length of each dimension and the number of elements in A and B. *[Usually given for 4 marks]*

♣ Ans:

Given, a 2D array A (-5:5, 3:33)

Here, Lower bound of the row, LB_R = -5

Upper bound of the row, UB_R = 5

Lower bound of the col, LB_C = 3

Upper bound of the col, UB_C = 33

Hence,

Number of Rows, R = 5 - (-5) + 1 = 11

Number of Columns, C = 33 - 3 + 1 = 31

Total number of elements = 31 * 11 = 341

3. Consider a 2D array A (1:5, 1:10)

a. Find the length of each dimension and number of elements in A.

- b. Suppose Base (A) = 300 and $w = 8$ words per memory cell for A. Find the row major address and column major address of the element A [3, 8].

♣ Ans:

a. Here,

Lower bound of the row, $LB_R = 1$

Upper bound of the row, $UB_R = 5$

Lower bound of the col, $LB_C = 1$

Upper bound of the col, $UB_C = 10$

Hence,

Number of Rows, $R = 5 - 1 + 1 = 5$

Number of Columns, $C = 10 - 1 + 1 = 10$

Total number of elements = $10 * 5 = 50$

b. Given,

Base (A) = 300

$w = 8$

Address of the element A [3, 8] in row major order:

$$\begin{aligned}\text{Loc (A [3, 8])} &= \text{Base (A)} + w [C (J - LB_R) + (K - LB_C)] \\ &= 300 + 8 [10 (3-1) + (8 - 1)] \\ &= 300 + 8 [20 + 7] = 300 + 216 = 516\end{aligned}$$

Address of the element A [3, 8] in column major order:

$$\begin{aligned}\text{Loc (A [3, 8])} &= \text{Base (A)} + w [R (K - LB_C) + (J - LB_R)] \\ &= 300 + 8 [5 (8 - 1) + (3 - 1)] \\ &= 300 + 8 [35 + 2] = 596\end{aligned}$$

4. Consider a 2D array A (-5:5, 3:33) and B (3:10, 1:15, 10:20) *[Usually given for 2 to 3 marks]*
- Find the length of each dimension and the number of elements in A and B
 - Suppose Base (B) = 400 and $w = 4$ words per memory location. Find the row effective indices E_1, E_2, E_3 and address of the element B [5, 10, 15] assuming B is sorted in row major order.
5. Let A be $n*n$ square matrix. Write a module which *[3 marks]*
- Find the number NUM of zero elements in A
 - Find the SUM of the elements above the diagonal i.e., elements A [I, J] where $I < J$

♣ Ans:

a.

Step 1: Set Num: = 0

Step 2: Repeat Steps 3 and 4 for $I = 1$ to n

Step 3: Repeat Step 4 for $J = 1$ to n

Step 4: If $A [I, J] = 0$, then Set Num: = Num + 1

Step 5: Write Num

Step 6: Exit

6. What is Multidimensional array?
7. What do you mean by *sparse matrix*? How a *lower triangular matrix* can be represented in computer memory without storing the 0s of the upper triangle? Explain. *[given for 2 marks]*